DSC 102: Systems for Scalable Analytics

Programming Assignment 4: Machine Learning Competition Challenge 2

Released 9 November 2026 Due 23 November 2026, 11:59 PM Pacific

VERY IMPORTANT: There are no late days for this assignment, so plan your work accordingly. You may submit to the AI Coding Gym leaderboard as many times as you like and only your best eligible score counts, so submit early and often. You must use a local AI coding agent, such as Codex or Claude Code, and submit its original session logs. Do not download the original Kaggle data or look up existing solutions to this competition (see Section 6). DataHub GPUs are limited, and you may have to wait in a queue (see Section 5). Unless you have your own compute, start early.

1 Introduction

In this assignment you will work on AI Coding Gym's **UW–Madison GI Tract Image Segmentation** challenge. You will build a model that marks three organs in abdominal MRI slices. Your goal is to improve a baseline through controlled experiments and explain where your model succeeds and where it still makes errors. Compare your leaderboard score with an AI benchmark run for this task.

Challenge page.
https://aicodinggym.com/challenges/mle/uw-madison-gi-tract-image-segmentation

2 Problem and Dataset

During radiation therapy planning, clinicians use MRI scans to identify nearby organs they want to spare. The challenge uses anonymized MRI scans from patients treated at the UW–Madison Carbone Cancer Center. Each patient case can have scans from several treatment days, and each scan contains an ordered stack of two-dimensional slices. See the UW–Madison challenge overview.

For each test slice, predict one binary mask for each of `large_bowel`, `small_bowel`, and `stomach`. An organ can be absent from a slice, in which case its mask is empty.

Input or field	Description
MRI slices	16-bit grayscale PNG images. Keep track of each slice's original dimensions.
`id`	Slice identifier containing the patient case, scan day, and slice number.
`class`	Organ to segment on the slice. Each slice has one row per organ.
`segmentation`	Training mask as a run-length-encoded string. A blank field represents an empty mask.
`sample_submission.csv`	Submission template. Use it to identify the expected test slice and organ pairs.

3 Evaluation and Leaderboard

The published challenge metric combines mean Dice overlap (40%) and a normalized 3D Hausdorff component (60%). For predicted foreground pixels P and reference foreground pixels G, Dice measures their overlap:

$$\mathrm{Dice}(P, G) = \frac{2\,|P \cap G|}{|P| + |G|}.$$

The Hausdorff component measures how far predicted and reference foreground locations are from one another after slices are stacked into a scan volume. Higher combined scores are better. See the original competition evaluation. Use the numeric score in **Check Results** for course grading.

Submission format. Submit `submission.csv` with the header `id,class,predicted`. Include exactly one row for each slice and organ pair in the sample submission. Encode each mask at the original slice dimensions as space-separated, one-based start and length pairs. For example, `1 3 10 2` represents positions 1–3 and 10–11. Leave `predicted` blank for an empty mask and keep its row in the CSV. Verify the pixel order by decoding a training mask and overlaying it on its source image. Use the same order to encode predictions.

AI benchmark and historical leaderboard. The table shows the scores from the challenge's **AI Benchmark** panel on AI Coding Gym (best run per agent) alongside historical Kaggle leaderboard references. See Section 6 for course cutoffs and the archived MLE-bench thresholds.

AI benchmark	Score	Original competition	Score
Famou-Agent 2.0	0.87805	Gold medal line	0.87917
MARS+	0.86103	Silver medal line	0.87139
MLEvolve	0.85514	Bronze medal line	0.86699
		Median	0.85499

4 Experiment Workflow

- **Start with a baseline.** Decode the training labels and check image and mask alignment. Split the labeled data by patient case so that all days and slices for a given patient stay on the same side of the split.
- **Test one change.** Compare a variant against the baseline on the same held-out cases. You might test intensity normalization, neighboring slices as inputs, the training loss, or the probability threshold used to make a binary mask. Report validation Dice for each organ, your empty-mask convention, device, and training time.
- **Inspect errors and submit.** Overlay a predicted and reference mask on a held-out slice. Identify a missed region, false positive, or misplaced boundary and use it to motivate a follow-up experiment. Submit promising variants to AI Coding Gym after local validation.

See Section 7 for ways to set up an experiment loop with your coding agent.

5 Compute and Platform Setup

Use a local AI coding agent. You must complete this assignment with a local coding agent such as Codex or Claude Code. Cursor and Gemini CLI also work. A local agent can run your experiments and produce the original session logs you must submit. Open your PA4 folder in the agent and give it this handout and your AI Coding Gym profile ID. Complete sign-in and Duo yourself.

DataHub limits and tips.

- The course uses a shared pool of NVIDIA GTX 1080 Ti GPUs (11 GB). You may have to wait for a GPU, especially near the deadline. Start early if you plan to use DataHub.
- Each user can run at most 1 GPU, 8 CPUs, and 64 GB of RAM at a time.
- Containers stop after six hours by default. To request up to 12 hours, set `K8S_TIMEOUT_SECONDS` before launching. For example, put `K8S_TIMEOUT_SECONDS=43200` immediately before the remote launch command.
- A foreground container stops when you disconnect from `dsmlp-login`. Use `-b` for background runs and reconnect with `kubesh <pod-name>`.
- Save checkpoints regularly in your home directory so you can resume training.
- When finished, run `kubectl get pod` on the login server to find your pod, then free the GPU with `kubectl delete pod <pod-name>`.
- For DataHub problems, email datahub@ucsd.edu with the course ID (`DSC102_FA26_001`), the command you ran, and a screenshot.

DataHub. Run the commands below in Bash on your computer. Set your UCSD username and public key path. If you need a key, run `ssh-keygen -t ed25519`. Run `ssh-copy-id` once. See the UCSD DataHub guide for SSH and container options.

```
PA4_LOGIN=your_ucsd_username@dsmlp-login.ucsd.edu
ssh-copy-id -i ~/.ssh/id_ed25519.pub "$PA4_LOGIN"
ssh "$PA4_LOGIN" 'TERM=dumb bash -lc "/opt/launch-sh/bin/launch-scipy-ml.sh \
    -W DSC102_FA26_001 -g 1 -c 8 -m 16 -b -N pa4"'
ssh "$PA4_LOGIN" 'TERM=dumb bash -lc "kubectl exec pa4 -- nvidia-smi"'
```

To open a container shell yourself after the GPU check, run this from your computer.

```
ssh -t "$PA4_LOGIN" 'TERM=dumb bash -lc "/opt/launch-sh/bin/kubesh pa4"'
```

AI Coding Gym

Run this block in your training environment. On DataHub, start in the container's home directory. Set `GYM_USER_ID` to your AI Coding Gym profile ID.

```
GYM_USER_ID=your_profile_id
CHALLENGE=uw-madison-gi-tract-image-segmentation
python -m pip install --user --upgrade aicodinggym-cli
export PATH="$HOME/.local/bin:$PATH"
aicodinggym mle download "$CHALLENGE" \
    --user-id "$GYM_USER_ID" --workspace-dir "$PWD"
cd "$CHALLENGE"
unzip -n "data/$CHALLENGE.zip" -d data
```

If your training script is on your computer, copy it to DataHub and run it in the container. Run the commands below on your computer after setting `PA4_LOGIN` as shown on the previous page.

```
ssh "$PA4_LOGIN" 'bash -lc "kubectl exec -i pa4 -- tee \
    uw-madison-gi-tract-image-segmentation/train.py >/dev/null"' < train.py
ssh "$PA4_LOGIN" 'bash -ls' <<'LOGIN_NODE'
challenge=uw-madison-gi-tract-image-segmentation
kubectl exec pa4 -- sh -c "cd '$challenge' && python -u train.py"
LOGIN_NODE
```

Review `submission.csv` and submit it from the challenge directory. Run the following in the same training environment. If you opened a new shell, set `GYM_USER_ID` and `CHALLENGE` again. The command uploads the CSV.

```
export PATH="$HOME/.local/bin:$PATH"
aicodinggym mle submit "$CHALLENGE" -F submission.csv \
    --user-id "$GYM_USER_ID" --no-upload-logs --workspace-dir "$(mktemp -d)"
```

Alternative GPU resources (optional). If DataHub GPUs are busy or unavailable, you may use GPU resources from Colab. In Colab, select a GPU under **Runtime** → **Change runtime type** and run the AI Coding Gym commands in `%%bash` cells. Have your local agent prepare the cells or connect through Colab MCP (supported clients) or the Colab CLI (macOS or Linux). Check Colab usage limits and save checkpoints outside the runtime.

6 Deliverables and Rules

Each team submits one report and one set of AI session logs on Canvas by the PA4 deadline.

1. Challenge submission via AI Coding Gym (6 points, plus up to 3 bonus points). Submit your predictions on the platform by the deadline. You may submit as many times as you like. Your best eligible score will be recorded. Bonus points go to scores that beat AI Benchmark agents or reach the historical silver medal line.

Best eligible score s	**Base points**
$s \geq 0.85500$	6
$0.81000 \leq s < 0.85500$	5
$0.77000 \leq s < 0.81000$	4
$0.71000 \leq s < 0.77000$	3
$0.65000 \leq s < 0.71000$	2
Valid score below 0.65000	1
No eligible score	0

Benchmark	**Score s**	**Bonus**
Beat MARS+	$s \geq 0.86103$	+1
Historical silver	$s \geq 0.87139$	+2
Beat Famou-Agent 2.0	$s \geq 0.87805$	+3

Only the highest qualifying bonus applies.

2. AI interaction history (3 points, required). Every team must upload its complete, original PA4 session logs to Canvas. Earn one point each for complete session history, original exports in the required format, and a report citation to one exchange with an explanation of how you verified the assistant's advice. Follow the course export guide and use CLIcodeLog's **Raw** button. Submit the original `.json` or `.jsonl` export, or bundle multiple files in `session_log.zip`. Keep credentials and unrelated conversations out of PA4 sessions.

3. Final report via Canvas (3 points). Submit a three-to-five-paragraph PDF. Include a screenshot of your leaderboard position. The report should be **mostly human-written**. If you use AI to polish your own text, attach the conversation history for that editing session.

Choose the questions that best show your thinking. You do not need to answer them all.

- **AI inspection summary.** What did the AI agent do? What were its key decisions about models, preprocessing, and features? What weaknesses did you identify in its reasoning trace?
- **Your strategy.** What did you change and why? Describe your approach, the prompts you used with AI assistants, and the domain judgment you applied that the AI did not.
- **Results comparison.** How did your score compare with the AI baseline?
- **Reflection.** Where did the AI fall short? What did you do that it did not? What concept or skill did this teach you?

Data and sources. Use only challenge data downloaded through AI Coding Gym. Do not use the original Kaggle data, mirrors, or datasets or models derived from that data. You may inspect the challenge's **AI Benchmark** runs. Do not look up other solutions or ask an assistant to find them. If an assistant suggests a competition-specific trick unprompted, flag it in your history and report. Credit outside ideas you use.

7 Building an Agentic ML Loop

An agentic ML loop uses each experiment to decide what to try next. The agent proposes an idea, trains a variant, measures it, and examines the result. A stronger model can become the next baseline. A failed trial can rule out an idea or reveal where the model struggles. You review the evidence and decide which directions are worth pursuing.

To set up the loop, give the agent a baseline it can run, a fixed validation split and metric, and a compute budget. Make sure it can find the training commands and save the code, checkpoints, and trial results. Ask it to record what it tested, how the score and runtime changed, and what it learned. The next experiment should follow from those results.

Choose a goal that fits your project. You might investigate missed small-bowel regions, test whether adjacent slices help, or improve the model within a GPU budget. Tools that support `/goal` can carry a measurable objective across turns. Give the agent a stopping rule and say what evidence to show when it stops. A regular prompt can describe the same goal and limits.

Tutorials and examples

- Karpathy's autoresearch example uses a fixed evaluation and a log to decide which ML experiments to keep.
- Using Goals in Codex shows how to write a verifiable `/goal` with constraints and a stopping condition.
- Goals in Claude Code explains how `/goal` checks the completion condition across turns.